Examination Result and Grace-Mark Validation:

Process marks across multiple subjects, apply subject-wise pass criteria, and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

Problem Statement

Develop a program to process marks across multiple subjects. The program must check the minimum passing marks for each subject and determine whether the student has passed or failed.

If the student fails a subject by a small number of marks, the program should apply a grace-mark policy, provided the student meets the eligibility conditions. Grace marks must not cause the final marks to exceed the maximum marks for any subject. Finally, the program should calculate the total marks, percentage, and final classification, and display the original marks, grace marks awarded, and final marks for each subject.

Python Code

```python
subjects = 5
pass_marks = 40
grace_limit = 5
max_marks = 100

original_marks = []
grace_marks = [0] * subjects

# Input marks for five subjects
for i in range(subjects):
    mark = int(input(f"Enter marks for Subject {i + 1}: "))
    original_marks.append(mark)

# Validate marks
if any(mark < 0 or mark > max_marks for mark in original_marks):
    print("\nInvalid marks! Marks must be between 0 and 100.")
else:
    final_marks = original_marks.copy()

    failed_subjects = [i for i in range(subjects)  if original_marks[i] < pass_marks]

    # Apply grace marks if exactly one subject failed and has 35-39 marks
    if len(failed_subjects) == 1:
        i = failed_subjects[0]
        if 35 <= original_marks[i] <= 39:
            required_grace = pass_marks - original_marks[i]
            if required_grace <= grace_limit:
                grace_marks[i] = required_grace
                final_marks[i] = min(original_marks[i] + required_grace, max_marks)

    total = sum(final_marks)
    percentage = (total / (subjects * max_marks)) * 100

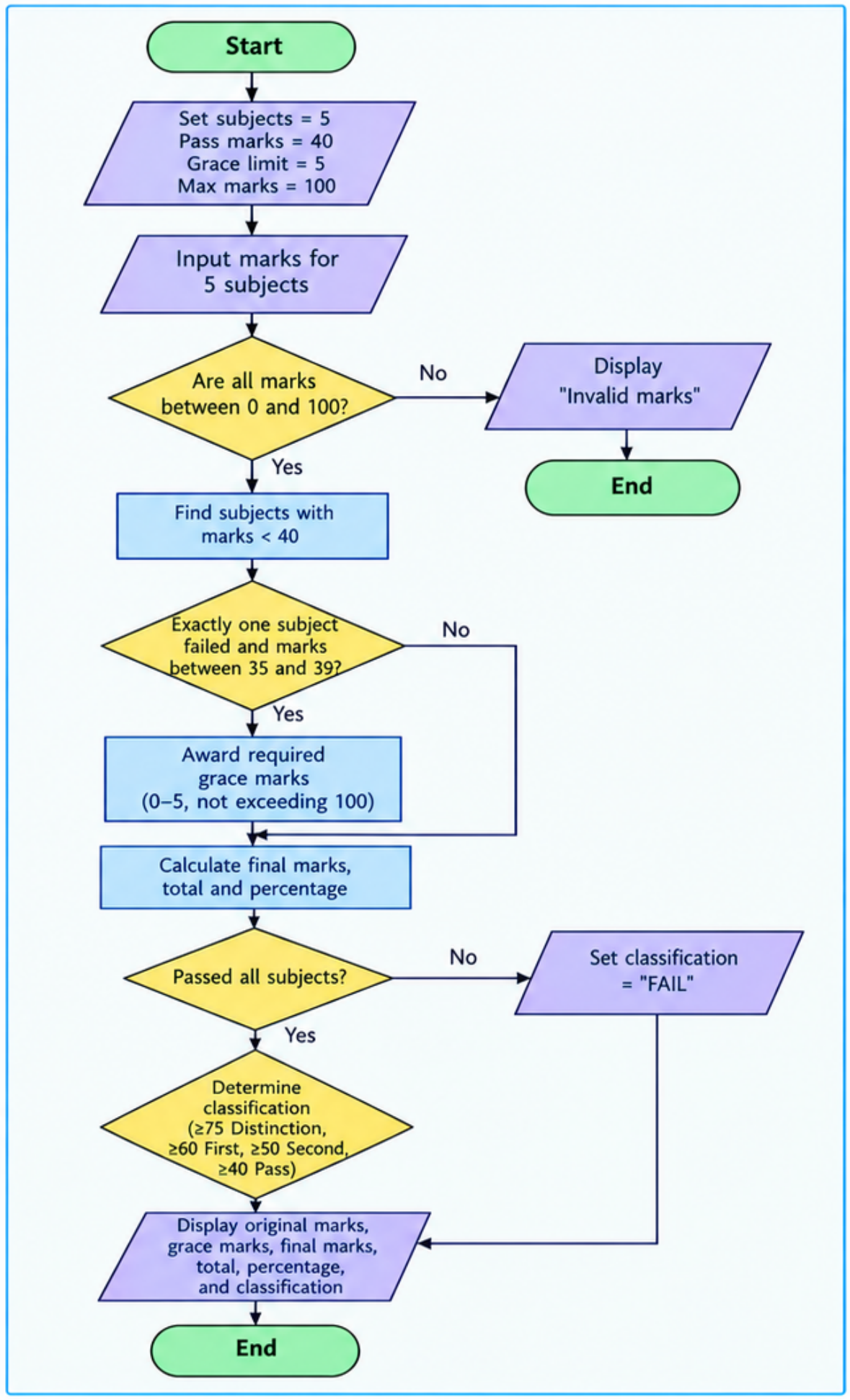
    if any(mark < pass_marks for mark in final_marks):
        classification = "FAIL"
    elif percentage >= 75:
        classification = "DISTINCTION"
    elif percentage >= 60:
        classification = "FIRST CLASS"
    elif percentage >= 50:
        classification = "SECOND CLASS"
    else:
        classification = "PASS"

    # Display result
    print("\n========== EXAMINATION RESULT ==========")
    print(f"Subject\tOriginal\tGrace\tFinal")
    for i in range(subjects):
        print(f"{i + 1}\t{original_marks[i]}\t{grace_marks[i]}\t{final_marks[i]}")
    print("-" * 45)
    print(f"Total Marks: {total} / {subjects * max_marks}")
    print(f"Percentage: {percentage:.2f}%")
    print("Classification:", classification)
```

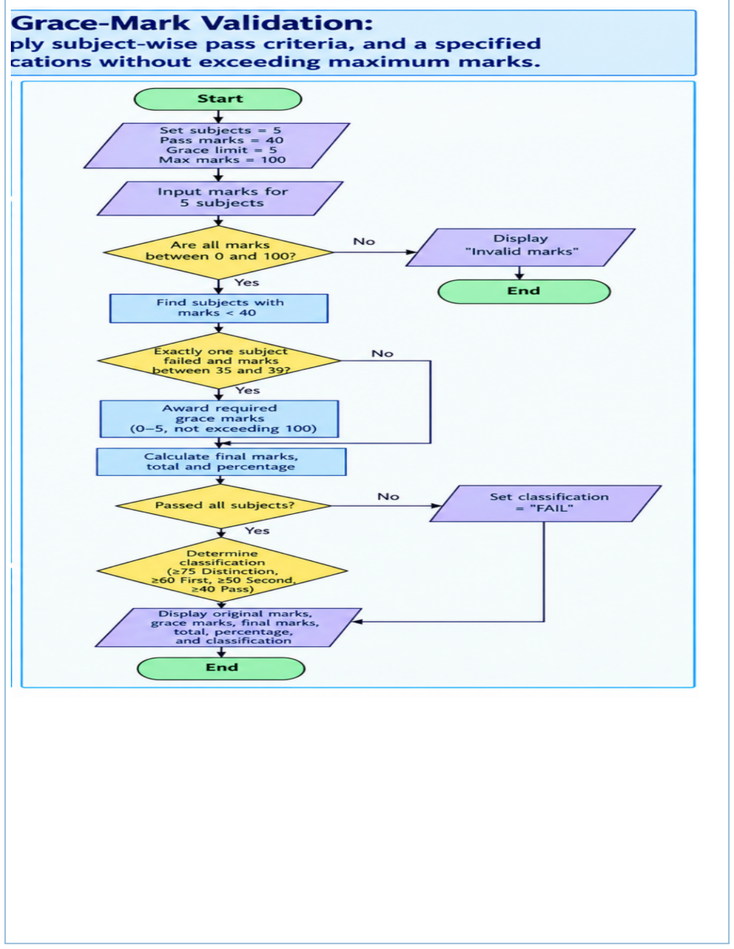
Program Output

```
Enter marks for Subject 1: 78
Enter marks for Subject 2: 65
Enter marks for Subject 3: 38
Enter marks for Subject 4: 82
Enter marks for Subject 5: 71

========== EXAMINATION RESULT ==========
Subject        Original        Grace          Final
    1             78             0             78
    2             65             0             65
    3             38             2             40
    4             82             0             82
    5             71             0             71
---------------------------------------------
Total Marks: 336 / 500
Percentage: 67.20%
Classification: FIRST CLASS
```



PROGRAM 1: Examination Result and Grace-Mark Validation



A. Algorithm

1. Start.

2. Set the number of subjects, pass mark, grace-mark limit and maximum mark.

3. Read the marks for every subject and validate that each mark is within 0 and the maximum mark.

4. Copy the original marks into a final-marks list and identify subjects below the pass mark.

5. If exactly one subject has failed and its mark is within the permitted grace range, calculate the marks required to pass.

6. If the required grace marks do not exceed the grace limit, award them without allowing the final mark to exceed the maximum.

7. Calculate total final marks, percentage and classification; display original, grace and final marks.

8. Stop.

B. Pseudocode

```
START
SET subjects, pass_mark, grace_limit, maximum_mark
INPUT marks for all subjects
IF any mark < 0 OR any mark > maximum_mark
DISPLAY "Invalid marks"
STOP
END IF
final_marks = copy(marks)
failed_subjects = indices where marks[index] < pass_mark
IF count(failed_subjects) = 1
i = the only failed subject
IF marks[i] is in the permitted grace range
required = pass_mark - marks[i]
IF required <= grace_limit
grace[i] = required
final_marks[i] = MIN(marks[i] + required, maximum_mark)
END IF
END IF
END IF
total = SUM(final_marks)
percentage = total / (number_of_subjects * maximum_mark) * 100
DETERMINE result/classification from final marks and percentage
DISPLAY marks, grace, final_marks, total, percentage, classification
STOP
```

2. Attendance Recovery Computation

Calculate attendance percentage and the minimum consecutive classes needed to meet an eligibility threshold.

Problem Statement

Given classes attended and total classes held, compute current attendance. For each future scenario, determine the smallest number of consecutive classes the student must attend to reach the required percentage. Validate inputs and report when recovery is impossible under the stated scenario.

Python Code

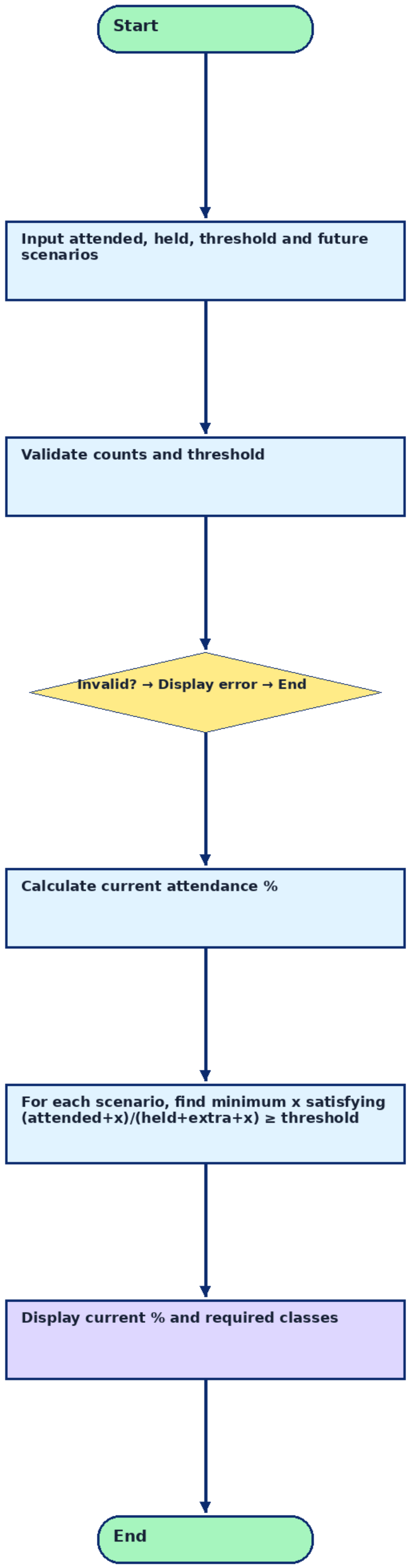
```
attended = int(input("Classes attended: "))
held = int(input("Classes held: "))
threshold = float(input("Eligibility percentage: "))
future_scenarios = [0, 5, 10]  # extra classes held and attended

if held <= 0 or attended < 0 or attended > held:
    print("Invalid attendance data")
elif not 0 < threshold <= 100:
    print("Invalid threshold")
else:
    current = attended / held * 100
    print(f"Current attendance: {current:.2f}%")
    for extra_held in future_scenarios:
        # x future classes are all attended
        x = 0
        while (attended + x) / (held + extra_held + x) * 100 < threshold:
            x += 1
            if x > 100000:
                break
        print(f"Scenario +{extra_held} held: attend {x} more")
```

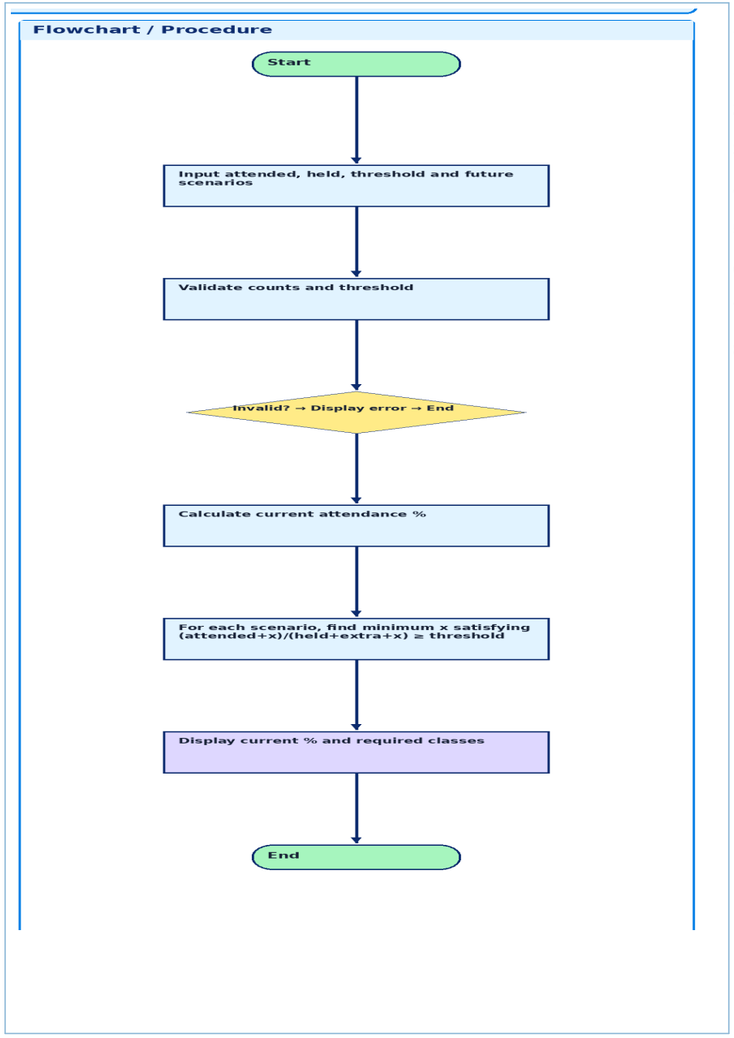
Sample Program Output

```
Classes attended: 36
Classes held: 50
Eligibility percentage: 75
Current attendance: 72.00%
Scenario +0 held: attend 6 more
Scenario +5 held: attend 21 more
Scenario +10 held: attend 36 more
```

Flowchart / Procedure



PROGRAM 2: Attendance Recovery Computation



A. Algorithm

1. Start.

2. Input classes attended, classes held and the required attendance percentage.

3. Validate that classes held is positive, attended is between zero and classes held, and the threshold is greater than zero and at most 100.

4. Calculate and display current attendance percentage.

5. For each future scenario, set the number of additional classes to zero.

6. Increase the additional classes one at a time while the projected attendance is below the threshold.

7. Display the minimum additional consecutive classes required for each scenario, or report if the search limit is exceeded.

8. Stop.

B. Pseudocode

```
START
INPUT attended, held, threshold
IF held <= 0 OR attended < 0 OR attended > held
DISPLAY "Invalid attendance data"
STOP
END IF
IF threshold <= 0 OR threshold > 100
DISPLAY "Invalid threshold"
STOP
END IF
current = attended / held * 100
DISPLAY current
FOR each extra_held scenario
x = 0
WHILE (attended + x) / (held + extra_held + x) * 100 < threshold
x = x + 1
IF x exceeds search limit
EXIT loop
END IF
END WHILE
DISPLAY scenario and x
END FOR
STOP
```

3. ATM Transaction Sequence Simulation

Process deposits and withdrawals while enforcing balance and transaction rules.

Problem Statement

Simulate a sequence of transactions starting from an opening balance. Accept valid deposits, reject non-positive amounts, reject withdrawals exceeding the balance or per-transaction limit, update the balance only for accepted transactions, and display a transaction summary.

Python Code

```
balance = float(input("Opening balance: "))
limit = float(input("Withdrawal limit: "))
transactions = [
    ("deposit", 500),
    ("withdraw", 200),
    ("withdraw", 900),
    ("deposit", -50)
]
accepted = rejected = 0

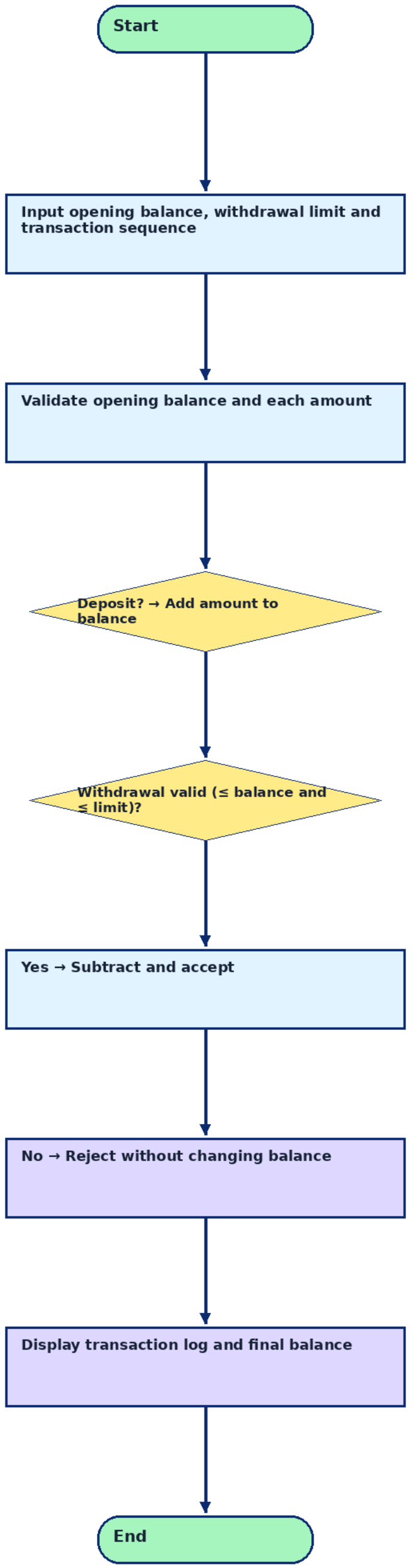
for kind, amount in transactions:
    if amount <= 0:
        print("Rejected:", kind, amount, "- invalid amount")
        rejected += 1
    elif kind == "deposit":
        balance += amount
        accepted += 1
        print("Deposit accepted:", amount)
    elif kind == "withdraw" and amount <= limit and amount <= balance:
        balance -= amount
        accepted += 1
        print("Withdrawal accepted:", amount)
    else:
        print("Rejected withdrawal:", amount)
        rejected += 1

print(f"Final balance: {balance:.2f}")
print("Accepted:", accepted, "Rejected:", rejected)
```

Sample Program Output

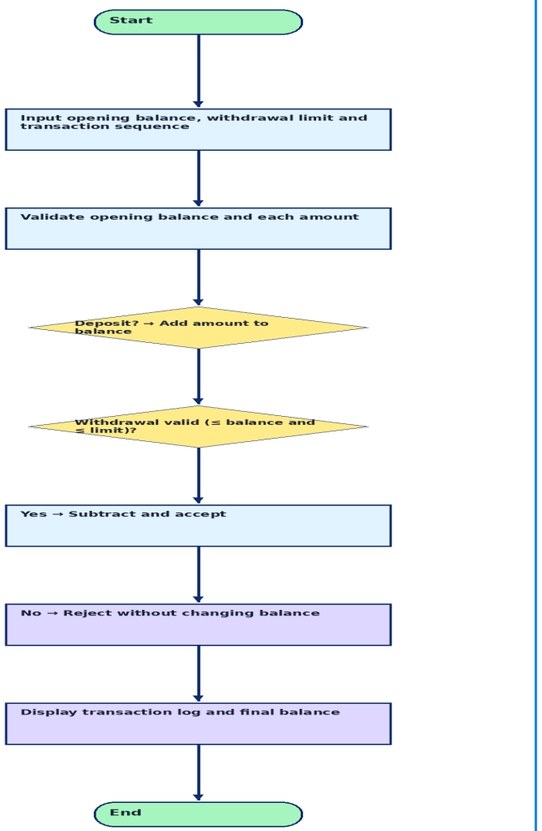
```
Opening balance: 1000.00
Withdrawal limit: 800.00
Deposit accepted: 500
Withdrawal accepted: 200
Rejected withdrawal: 900
Rejected: deposit -50 - invalid amount
Final balance: 1300.00
Accepted: 2 Rejected: 2
```

Flowchart / Procedure



PROGRAM 3: ATM Transaction Sequence Simulation

Flowchart / Procedure



A. Algorithm

1. Start.

2. Input opening balance, withdrawal limit and the sequence of transactions.

3. Validate the opening balance and withdrawal limit.

4. Set accepted and rejected transaction counters to zero.

5. For each transaction, reject an amount that is not positive.

6. For a deposit, add the amount to the balance and increase the accepted counter.

7. For a withdrawal, accept only when the amount is within both the transaction limit and current balance; otherwise reject it.

8. Display each transaction result, final balance and the accepted/rejected totals.

9. Stop.

B. Pseudocode

```
START
INPUT opening_balance, withdrawal_limit, transactions
balance = opening_balance
accepted = 0
rejected = 0
FOR each (type, amount) in transactions
IF amount <= 0
DISPLAY invalid amount
rejected = rejected + 1
ELSE IF type = "deposit"
balance = balance + amount
accepted = accepted + 1
DISPLAY deposit accepted
ELSE IF type = "withdraw"
IF amount <= withdrawal_limit AND amount <= balance
balance = balance - amount
accepted = accepted + 1
DISPLAY withdrawal accepted
ELSE
rejected = rejected + 1
DISPLAY withdrawal rejected
END IF
ELSE
rejected = rejected + 1
DISPLAY unknown transaction
END IF
END FOR
DISPLAY balance, accepted, rejected
STOP
```

4. Warehouse Inventory Reconciliation

Compare expected and actual stock and calculate replenishment needs for multiple products.

Problem Statement

For each product, reconcile opening stock plus incoming units minus dispatched units against the recorded closing stock. Flag discrepancies, prevent negative quantities, and calculate replenishment needed to reach a target stock level.

Python Code

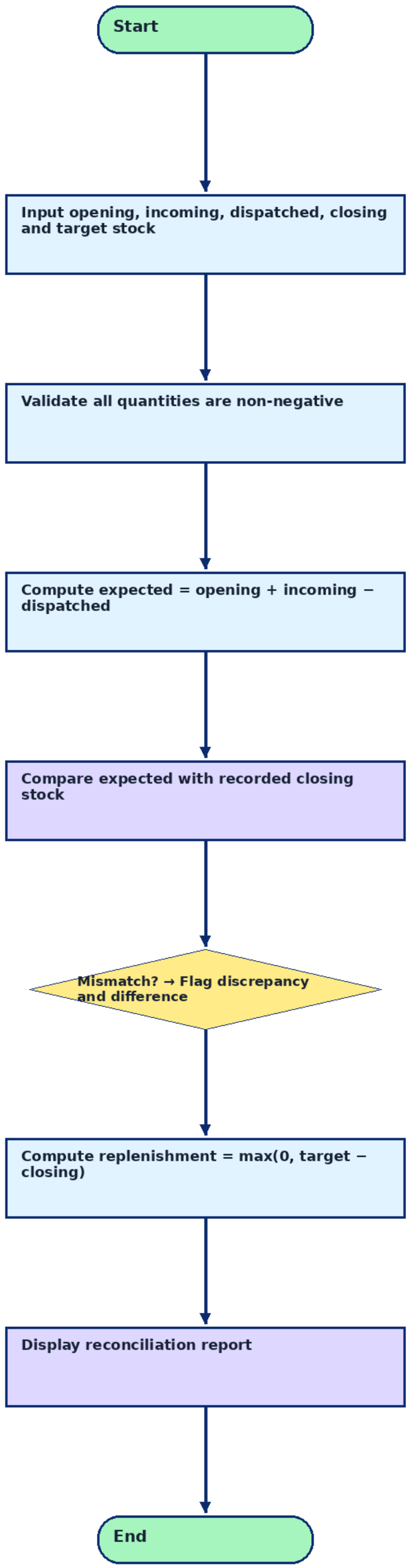
```
products = [
    {"name": "Bolts", "open": 100, "in": 40, "out": 90,
     "closing": 45, "target": 80},
    {"name": "Nuts", "open": 60, "in": 10, "out": 25,
     "closing": 40, "target": 50}
]

for p in products:
    expected = p["open"] + p["in"] - p["out"]
    discrepancy = p["closing"] - expected
    replenish = max(0, p["target"] - p["closing"])
    status = "MATCH" if discrepancy == 0 else "DISCREPANCY"
    print(p["name"], status,
          "expected=", expected,
          "recorded=", p["closing"],
          "difference=", discrepancy,
          "replenish=", replenish)
```

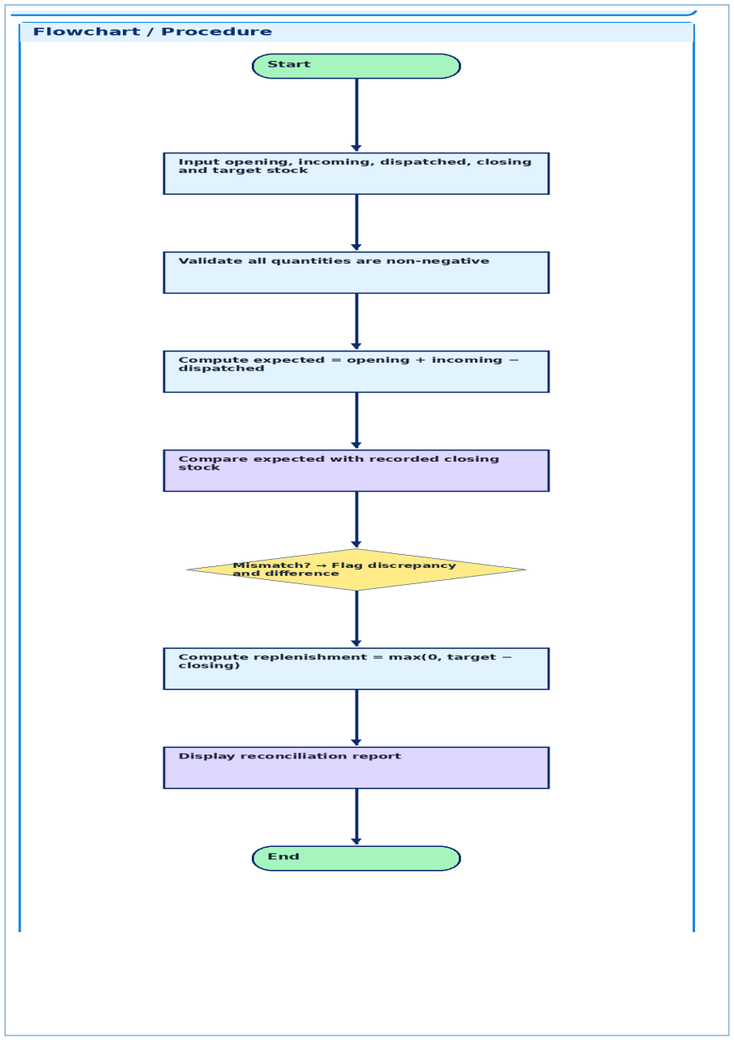
Sample Program Output

```
Bolts DISCREPANCY expected= 50 recorded= 45
difference= -5 replenish= 35
Nuts DISCREPANCY expected= 45 recorded= 40
difference= -5 replenish= 10
```

Flowchart / Procedure



PROGRAM 4: Warehouse Inventory Reconciliation



A. Algorithm

1. Start.

2. Read each product's opening stock, incoming units, dispatched units, recorded closing stock and target stock.

3. Validate that all quantities are non-negative.

4. Calculate expected stock as opening stock plus incoming units minus dispatched units.

5. Compare expected stock with recorded closing stock and calculate the discrepancy.

6. Calculate replenishment as the greater of zero and target stock minus recorded closing stock.

7. Display the product name, match/discrepancy status, expected stock, recorded stock, difference and replenishment quantity.

8. Stop.

B. Pseudocode

```
START
INPUT product records
FOR each product
READ opening, incoming, dispatched, recorded_closing, target
IF any quantity is negative
DISPLAY invalid product data
CONTINUE to next product
END IF
expected = opening + incoming - dispatched
difference = recorded_closing - expected
IF difference = 0
status = "MATCH"
ELSE
status = "DISCREPANCY"
END IF
replenishment = MAX(0, target - recorded_closing)
DISPLAY product, status, expected, recorded_closing,
difference, replenishment
END FOR
STOP
```

5. Smart Parking Fee Management

Compute parking charges from timestamps, vehicle type, daily caps and penalties.

Problem Statement

Validate entry and exit times, calculate parked duration including overnight stays, apply vehicle-specific hourly rates and daily maximums, and add any applicable penalty. Reject invalid or reversed timestamps and show the billing breakdown.

Python Code

```
from datetime import datetime, timedelta
from math import ceil

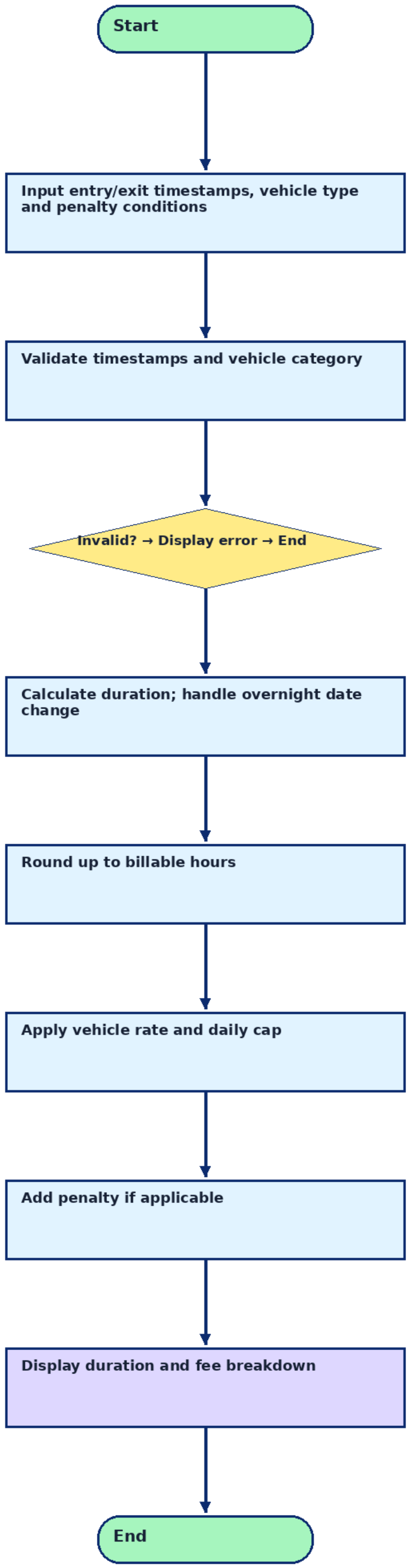
entry = datetime.strptime("2026-10-23 22:30", "%Y-%m-%d %H:%M")
exit_time = datetime.strptime("2026-10-24 02:10", "%Y-%m-%d %H:%M")
vehicle = "car"
rates = {"bike": 20, "car": 40, "van": 60}
daily_caps = {"bike": 100, "car": 200, "van": 300}
penalty = 0

if exit_time < entry or vehicle not in rates:
    print("Invalid parking input")
else:
    minutes = int((exit_time - entry).total_seconds() / 60)
    hours = ceil(minutes / 60)
    raw_fee = hours * rates[vehicle]
    days = max(1, ceil(minutes / 1440))
    fee = min(raw_fee, daily_caps[vehicle] * days) + penalty
    print("Duration:", hours, "billable hours")
    print("Base fee:", min(raw_fee, daily_caps[vehicle] * days))
    print("Penalty:", penalty)
    print("Total fee:", fee)
```

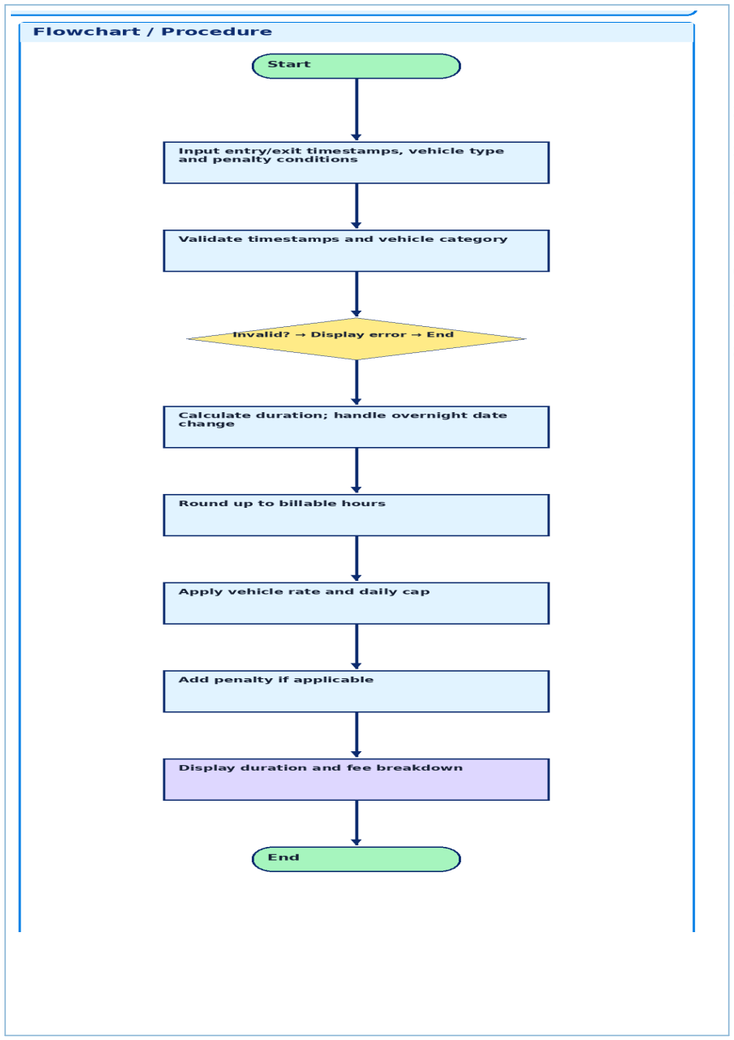
Sample Program Output

```
Duration: 4 billable hours
Base fee: 160
Penalty: 0
Total fee: 160
Example uses car rate = 40 per hour.
```

Flowchart / Procedure



PROGRAM 5: Smart Parking Fee Management



A. Algorithm

1. Start.

2. Input vehicle type, entry timestamp, exit timestamp and applicable penalty.

3. Validate that the vehicle type is supported and the exit timestamp is not earlier than the entry timestamp.

4. Calculate the parked duration in minutes, then round up to whole billed hours.

5. Select the hourly rate and daily cap for the vehicle type.

6. Calculate the time-based charge and apply the daily maximum rule.

7. Add any applicable penalty and display the duration and billing breakdown.

8. Stop.

B. Pseudocode

```
START
INPUT vehicle_type, entry_time, exit_time, penalty
IF vehicle_type is not supported OR exit_time < entry_time
DISPLAY "Invalid parking input"
STOP
END IF
minutes = duration_in_minutes(exit_time - entry_time)
hours = CEILING(minutes / 60)
rate = hourly_rate[vehicle_type]
daily_cap = cap[vehicle_type]
base_charge = calculate charge from hours and rate
parking_charge = APPLY daily cap to base_charge
total = parking_charge + penalty
DISPLAY minutes, hours, parking_charge, penalty, total
STOP
```

6. Multi-Day Water Tank Simulation

Track tank levels over time and detect overflow, shortage and threshold crossings.

Problem Statement

Starting with an initial water level, process each interval's inflow and outflow. Report overflow above tank capacity, shortage below zero, clamp the physical level to the valid range, and identify the first interval when the level reaches a target threshold.

Python Code

```
capacity = 1000
level = 600
threshold = 850
flows = [(200, 100), (100, 250), (300, 50), (50, 80)]
first_reached = None

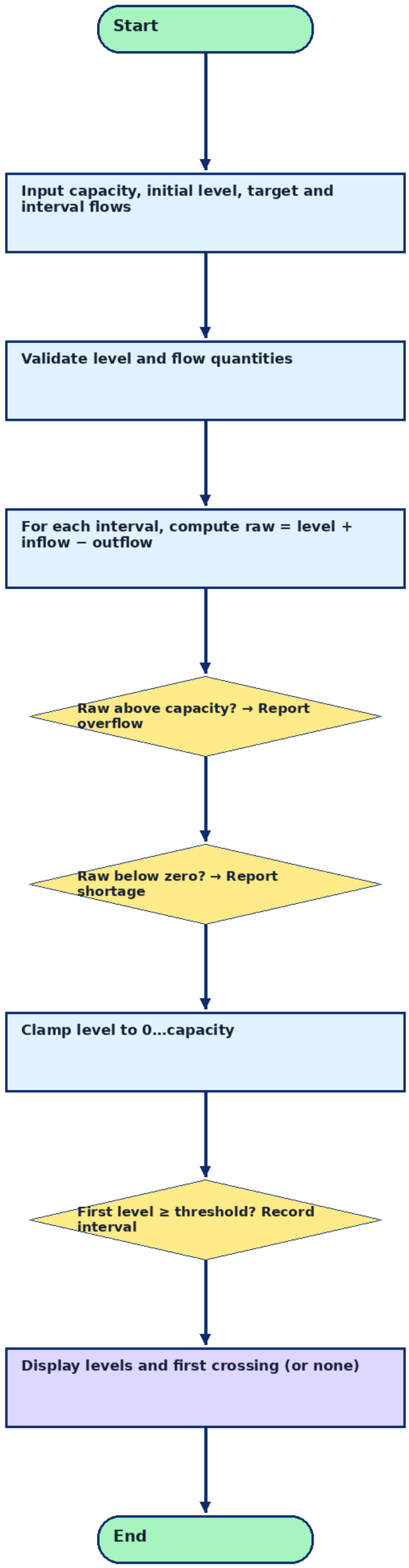
for day, (inflow, outflow) in enumerate(flows, start=1):
    raw = level + inflow - outflow
    if raw > capacity:
        print(f"Day {day}: overflow {raw - capacity}")
    if raw < 0:
        print(f"Day {day}: shortage {-raw}")
    level = min(capacity, max(0, raw))
    print(f"Day {day}: level={level}")
    if first_reached is None and level >= threshold:
        first_reached = day

print("First threshold interval:", first_reached)
```

Sample Program Output

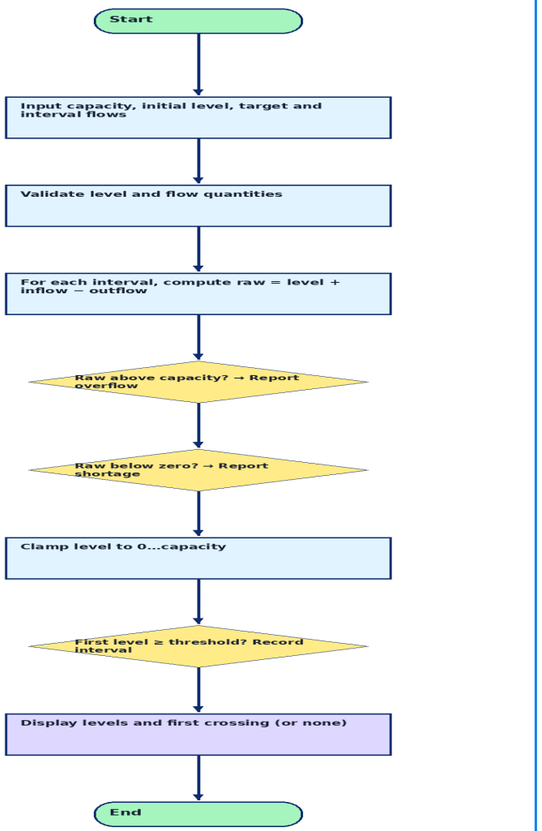
```
Day 1: level=700
Day 2: level=550
Day 3: level=800
Day 4: level=770
First threshold interval: None
```

Flowchart / Procedure



PROGRAM 6: Multi-Day Water Tank Simulation

Flowchart / Procedure



A. Algorithm

1. Start.

2. Set tank capacity, initial water level, target threshold and the sequence of daily inflow/outflow values.

3. Validate the initial level and flow values.

4. For each day, calculate the raw level as previous level plus inflow minus outflow.

5. If the raw level exceeds capacity, report overflow; if it is below zero, report shortage.

6. Clamp the physical tank level between zero and capacity.

7. If the target threshold has been reached for the first time, record that day.

8. Display the level and any overflow/shortage message for each day, then display the first threshold day if reached.

9. Stop.

B. Pseudocode

```
START
SET capacity, level, target_threshold, flows
first_reached = NONE
FOR each day with (inflow, outflow)
raw_level = level + inflow - outflow
IF raw_level > capacity
DISPLAY overflow amount = raw_level - capacity
ELSE IF raw_level < 0
DISPLAY shortage amount = 0 - raw_level
END IF
level = MIN(capacity, MAX(0, raw_level))
DISPLAY day and level
IF first_reached is NONE AND level >= target_threshold
first_reached = day
END IF
END FOR
IF first_reached is not NONE
DISPLAY first_reached
ELSE
DISPLAY target not reached
END IF
STOP
```

7. Student Performance Trend Analysis

Compare exam scores, identify changes and weak subjects, and classify performance.

Problem Statement

Read scores for several subjects across multiple examinations. Calculate score changes between exams, identify improvement or decline, mark subjects that remain below a defined threshold in every exam, and assign a classification using the overall average.

Python Code

```
scores = {
    "Math": [58, 66, 72],
    "Science": [70, 65, 62],
    "English": [45, 48, 49]
}
weak_limit = 50

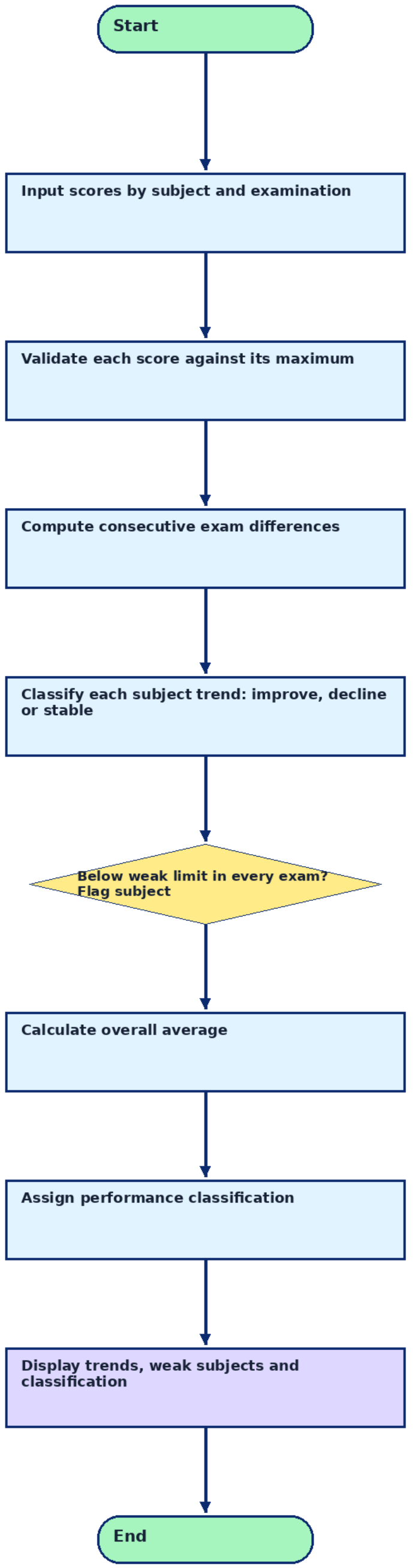
for subject, marks in scores.items():
    changes = [marks[i] - marks[i-1]
               for i in range(1, len(marks))]
    weak = all(m < weak_limit for m in marks)
    trend = "improving" if changes and changes[-1] > 0 else (
        "declining" if changes and changes[-1] < 0 else "stable")
    print(subject, "changes=", changes,
          "trend=", trend, "consistently weak=", weak)

all_marks = [m for values in scores.values() for m in values]
avg = sum(all_marks) / len(all_marks)
grade = "Excellent" if avg >= 75 else (
    "Good" if avg >= 60 else "Needs Improvement")
print(f"Average: {avg:.2f}; Classification: {grade}")
```

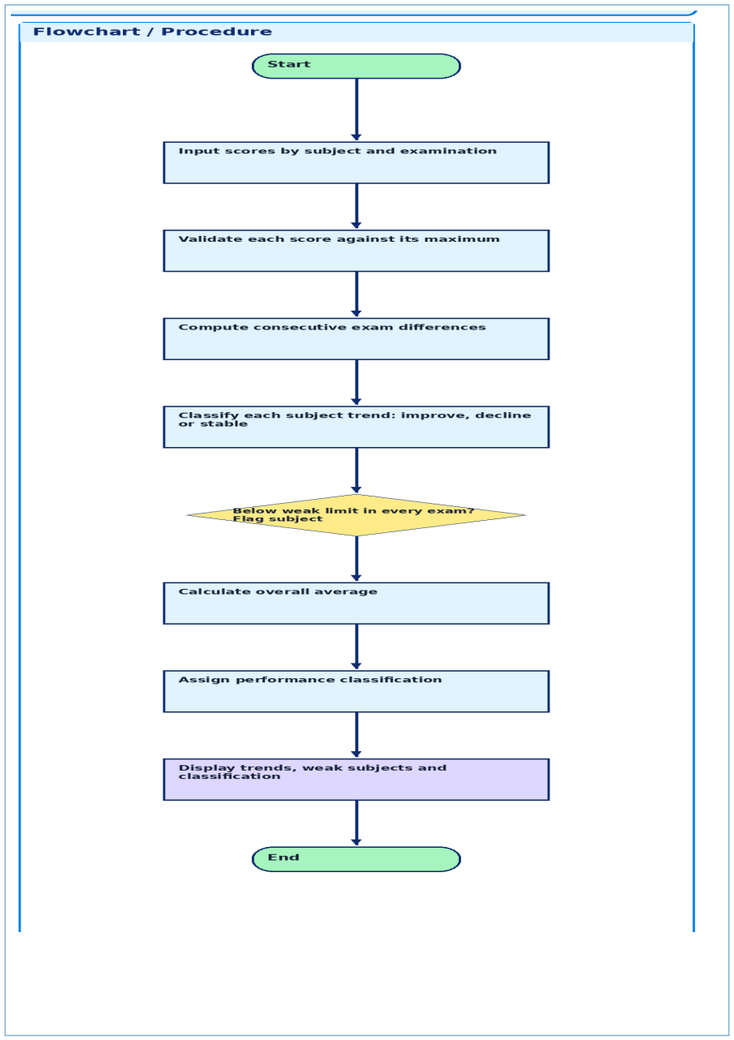
Sample Program Output

```
Math changes= [8, 6] trend= improving consistently weak= False
Science changes= [-5, -3] trend= declining consistently weak= False
English changes= [3, 1] trend= improving consistently weak= True
Average: 59.44; Classification: Needs Improvement
```

Flowchart / Procedure



PROGRAM 7: Student Performance Trend Analysis



A. Algorithm

1. Start.

2. Input marks for each subject across multiple examinations and set the weak-subject threshold.

3. Validate that all marks are within the permitted range.

4. For each subject, calculate the change in marks between consecutive examinations.

5. Check whether every examination mark for the subject is below the weak-subject threshold.

6. Classify the subject trend as improving, declining or stable based on the latest change.

7. Calculate the overall average and assign the overall performance classification using the program's defined bands.

8. Display subject-wise changes, weak-subject flags, trends and overall classification.

9. Stop.

B. Pseudocode

```
START
INPUT subject marks across examinations
SET weak_mark_limit and classification bands
VALIDATE all marks
FOR each subject
changes = differences between consecutive exam marks
weak = ALL marks for subject < weak_mark_limit
IF latest change > 0
trend = "improving"
ELSE IF latest change < 0
trend = "declining"
ELSE
trend = "stable"
END IF
DISPLAY subject, changes, weak, trend
END FOR
average = average of all marks
classification = category selected from average and defined bands
DISPLAY average and classification
STOP
```

8. Fuel Consumption and Route Comparison

Estimate fuel, cost and reserve feasibility for multiple vehicle choices.

Problem Statement

For each vehicle and route option, calculate trip fuel from distance and mileage, add reserve fuel, compute journey cost using fuel price, and check whether the vehicle's available fuel or budget is sufficient. Select the lowest-cost feasible option.

Python Code

```
distance_km = 320
fuel_price = 105
reserve_pct = 10
budget = 3000
vehicles = {
    "Car A": {"mileage": 16, "fuel_available": 25},
    "Car B": {"mileage": 20, "fuel_available": 20},
    "Van":   {"mileage": 10, "fuel_available": 40}
}
feasible = []

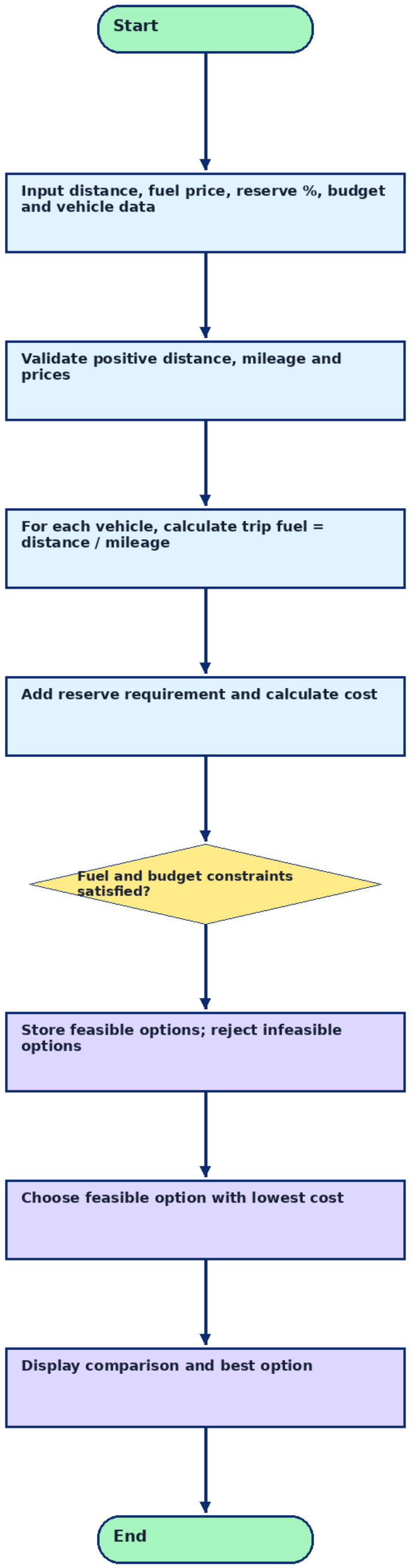
for name, v in vehicles.items():
    trip = distance_km / v["mileage"]
    needed = trip * (1 + reserve_pct / 100)
    cost = trip * fuel_price
    ok = needed <= v["fuel_available"] and cost <= budget
    print(name, f"trip={trip:.2f}L",
          f"reserve={needed:.2f}L", f"cost={cost:.2f}", ok)
    if ok:
        feasible.append((cost, name))

print("Lowest-cost feasible:",
      min(feasible)[1] if feasible else "None")
```

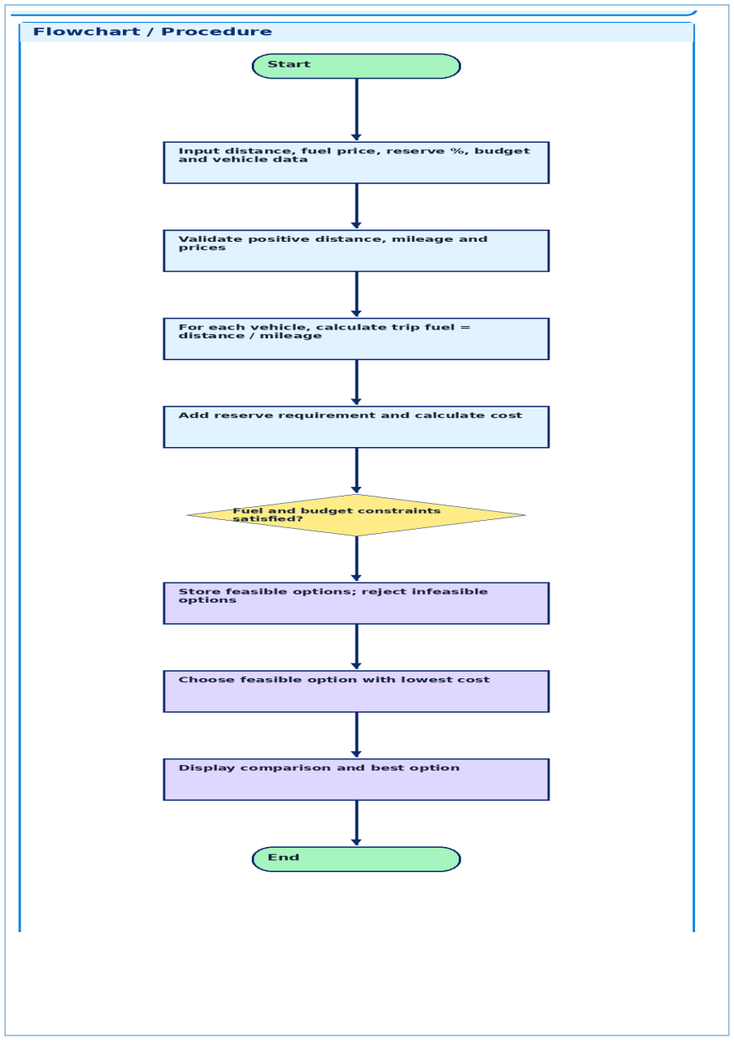
Sample Program Output

```
Car A trip=20.00L reserve=22.00L cost=2100.00 True
Car B trip=16.00L reserve=17.60L cost=1680.00 True
Van trip=32.00L reserve=35.20L cost=3360.00 False
Lowest-cost feasible: Car B
```

Flowchart / Procedure



PROGRAM 8: Fuel Consumption and Route Comparison



A. Algorithm

1. Start.

2. Input route distance, fuel price, reserve percentage, budget and vehicle details.

3. Validate distance, mileage, fuel price, reserve and budget values.

4. For each vehicle, calculate trip fuel as distance divided by mileage.

5. Add reserve fuel and calculate trip cost using the fuel price.

6. Check whether available fuel and budget are sufficient.

7. Mark each option as feasible or infeasible.

8. Select and display the lowest-cost feasible vehicle option, or report that none is feasible.

9. Stop.

B. Pseudocode

```
START
INPUT distance, fuel_price, reserve_percent, budget, vehicles
VALIDATE all input values
feasible_options = empty list
FOR each vehicle
trip_fuel = distance / vehicle.mileage
required_fuel = trip_fuel * (1 + reserve_percent / 100)
trip_cost = trip_fuel * fuel_price
IF required_fuel <= vehicle.fuel_available AND trip_cost <= budget
ADD vehicle and trip_cost to feasible_options
END IF
END FOR
IF feasible_options is not empty
best = feasible option with minimum trip_cost
DISPLAY best vehicle, required_fuel, trip_cost
ELSE
DISPLAY "No feasible vehicle option"
END IF
STOP
```

9. Multi-Product Shopping Budget Constraint

Apply quantity pricing and discounts, then reduce purchases to fit the budget.

Problem Statement

Calculate each basket line using unit price and quantity, apply qualifying discounts, and compare the total with a fixed budget. If over budget, reduce quantities according to a defined priority until the basket fits; report removed units and the final total.

Python Code

```
budget = 2500
basket = [
    {"name": "Notebook", "price": 100, "qty": 8},
    {"name": "Pen pack", "price": 80, "qty": 6},
    {"name": "Bag", "price": 900, "qty": 2}
]

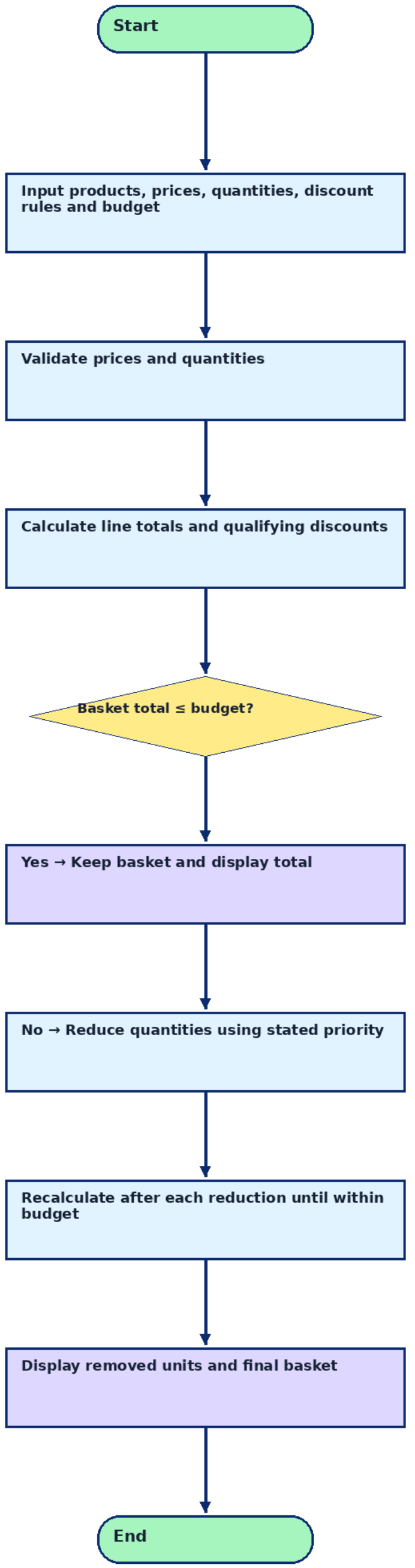
def total_cost(items):
    total = 0
    for item in items:
        line = item["price"] * item["qty"]
        discount = 0.10 * line if item["qty"] >= 6 else 0
        total += line - discount
    return total

print("Initial total:", total_cost(basket))
# Reduce lowest-priority items first: bag, pen pack, notebook
for name in ["Bag", "Pen pack", "Notebook"]:
    item = next(x for x in basket if x["name"] == name)
    while item["qty"] > 0 and total_cost(basket) > budget:
        item["qty"] -= 1
print("Final basket:", basket)
print("Final total:", total_cost(basket))
```

Sample Program Output

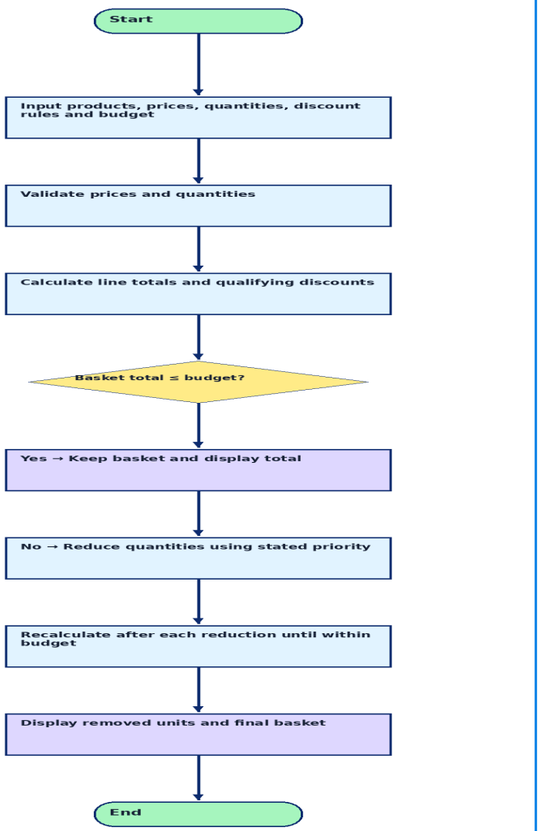
```
Initial total: 2760.00
Final basket:
Notebook: 8 units
Pen pack: 6 units
Bag: 1 unit
Final total: 1860.00
Within budget: Yes
```

Flowchart / Procedure



PROGRAM 9: Multi-Product Shopping Budget Constraint

Flowchart / Procedure



A. Algorithm

1. Start.

2. Input the shopping budget, product prices, quantities, discount rules and reduction priority.

3. Validate that prices, quantities and budget are non-negative.

4. Calculate each line amount and apply the qualifying quantity discount.

5. Add all discounted line totals to obtain the basket total.

6. If the total exceeds the budget, reduce quantities according to the defined priority and recalculate the basket.

7. Repeat until the total fits the budget or no further quantity can be reduced.

8. Display removed units, final quantities and final basket total.

9. Stop.

B. Pseudocode

```
START
INPUT budget, products, discount_rules, reduction_priority
VALIDATE prices, quantities and budget
FUNCTION calculate_total(products)
total = 0
FOR each product
line = product.price * product.quantity
discount = APPLY qualifying discount rule to line
total = total + line - discount
END FOR
RETURN total
END FUNCTION
total = calculate_total(products)
WHILE total > budget AND a quantity can be reduced
product = next reducible product by priority
reduce product quantity
record removed unit
total = calculate_total(products)
END WHILE
DISPLAY final quantities, removed units, total
IF total > budget
DISPLAY "Budget cannot be met with remaining quantities"
END IF
STOP
```

10. University Admission Decision Engine

Check course eligibility, rank qualified applicants and resolve ties consistently.

Problem Statement

Evaluate applicants using required subject marks, aggregate percentage and course-specific thresholds. Reject candidates who fail any eligibility rule. Rank eligible candidates by aggregate score, then by specified subject score and application ID as tie-breakers.

Python Code

```
courses = {
    "Computer Science": {"aggregate": 75, "math": 70},
    "Commerce": {"aggregate": 65, "math": 0}
}
applicants = [
    {"id": "A12", "course": "Computer Science",
     "aggregate": 82, "math": 78},
    {"id": "A07", "course": "Computer Science",
     "aggregate": 82, "math": 85},
    {"id": "B03", "course": "Commerce",
     "aggregate": 68, "math": 55}
]
eligible = []

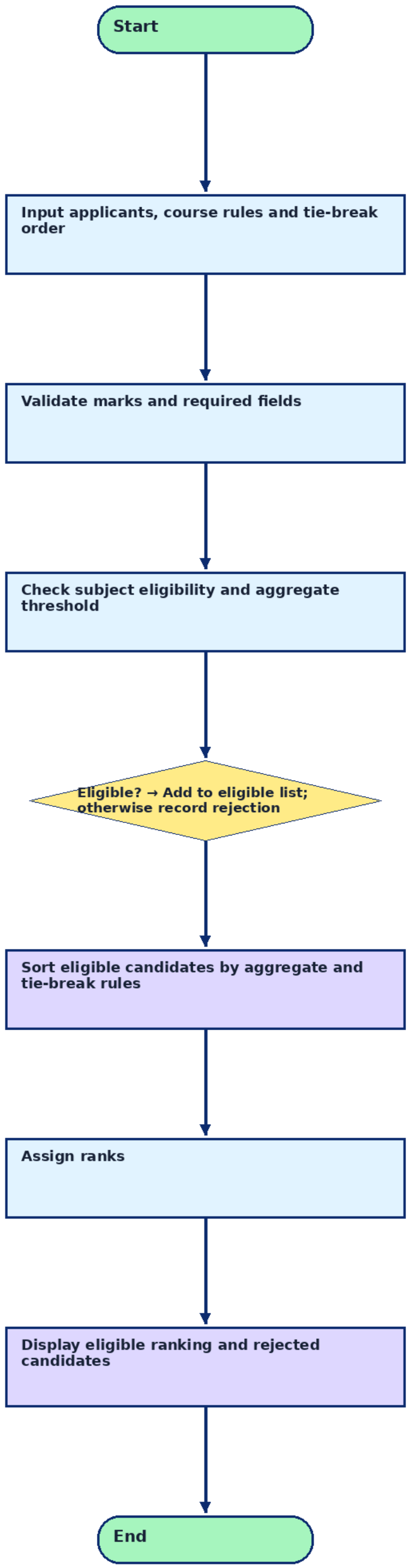
for a in applicants:
    rule = courses[a["course"]]
    ok = (a["aggregate"] >= rule["aggregate"] and
          a["math"] >= rule["math"])
    print(a["id"], "Eligible" if ok else "Rejected")
    if ok:
        eligible.append(a)

eligible.sort(key=lambda a: (
    a["course"], -a["aggregate"], -a["math"], a["id"]))
print("Ranked eligible list:")
for rank, a in enumerate(eligible, start=1):
    print(rank, a["course"], a["id"], a["aggregate"], a["math"])
```

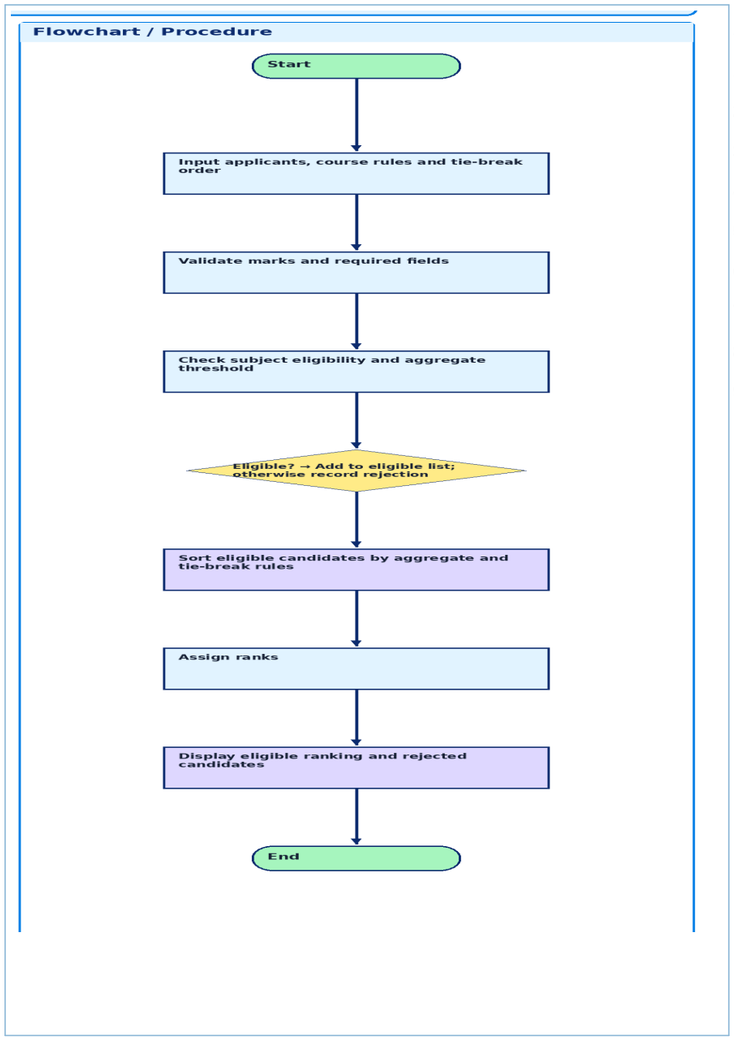
Sample Program Output

```
A12 Eligible
A07 Eligible
B03 Eligible
Ranked eligible list:
1 Computer Science A07 82 85
2 Computer Science A12 82 78
3 Commerce B03 68 55
Ranking is course-wise; subject score breaks aggregate ties.
```

Flowchart / Procedure



PROGRAM 10: University Admission Decision Engine



A. Algorithm

1. Start.

2. Input course eligibility requirements and applicant records.

3. Validate applicant marks and required fields.

4. For each applicant, identify the selected course and compare aggregate and required subject marks with that course's thresholds.

5. Reject applicants who do not meet every eligibility condition.

6. Collect applicants who meet all conditions.

7. Rank eligible applicants by aggregate score, then specified subject score, then application ID as the tie-breaker.

8. Display eligibility decisions and the ordered merit list.

9. Stop.

B. Pseudocode

```
START
INPUT course_requirements and applicants
VALIDATE applicant records
eligible = empty list
FOR each applicant
requirement = course_requirements[applicant.course]
IF applicant.aggregate >= requirement.aggregate
AND applicant.subject_mark >= requirement.subject_threshold
ADD applicant to eligible
DISPLAY applicant as eligible
ELSE
DISPLAY applicant as not eligible
END IF
END FOR
SORT eligible by:
aggregate descending,
specified subject mark descending,
application ID ascending
DISPLAY ranked eligible list
STOP
```